COMPRENDRE
L’INTELLIGENCE ARTIFICIELLE

Le guide complet pour débuter, comprendre les systèmes et construire ses premiers projets

Des neurones artificiels aux API, du prompting au MCP, des modèles aux agents autonomes.

MANUEL DE FORMATION • NIVEAU DÉBUTANT À INTERMÉDIAIRE

Objectif : passer de « je ne sais pas comment ça marche » à « je comprends les briques, leurs limites et comment les assembler ».

Version pédagogique — octobre 2026

Comment utiliser ce manuel

Ce document est conçu comme une formation progressive. Il peut se lire dans l'ordre ou servir de référence quand un terme technique apparaît.

- Lire activement — à la fin de chaque chapitre, reformule le concept avec tes propres mots.
- Dessiner les systèmes — un schéma simple aide souvent davantage qu'une longue définition.
- Tester rapidement — chaque notion importante est accompagnée d'un exemple ou d'un exercice.
- Vérifier les informations — les modèles peuvent se tromper ; une réponse fluide n'est pas automatiquement vraie.
- Construire par étapes — commence par un assistant simple avant d'ajouter des outils, une mémoire ou un agent.

Prérequis

Aucun prérequis en programmation n'est nécessaire pour les premières parties. Les chapitres techniques utilisent des analogies avant d'introduire le vocabulaire professionnel. Pour les exercices de construction, savoir lire un peu de code est utile, mais pas obligatoire.

Carte de la formation

Bloc	Tu apprendras à…
1. Fondamentaux	expliquer l'IA, les réseaux de neurones, l'apprentissage et les LLM.
2. Vocabulaire	reconnaître les 50 mots essentiels et les employer correctement.
3. Architecture	comprendre frontend, backend, serveur, API, base de données, RAG et MCP.
4. Usage	rédiger des prompts, choisir un modèle et vérifier les réponses.
5. Pratique	concevoir un assistant, évaluer sa qualité et progresser sur 30 jours.

Règle d'or : comprendre le rôle de chaque composant avant d'apprendre son nom par cœur.

1. Qu’est-ce que l’intelligence artificielle ?

Définition, exemples, limites et distinctions importantes

L’intelligence artificielle (IA) regroupe des méthodes qui permettent à des systèmes informatiques d’effectuer des tâches habituellement associées à certaines capacités humaines : reconnaître, classer, prédire, générer, traduire, planifier ou aider à décider.

L’IA n’est pas un logiciel unique. Un filtre anti-spam, un système de recommandation, un modèle de vision et un assistant conversationnel peuvent tous relever de l’IA, tout en utilisant des méthodes différentes.

Trois notions à distinguer

Notion	Sens simple	Exemple
Intelligence artificielle	Domaine général des systèmes réalisant des tâches intelligentes.	Assistant qui analyse une demande.
Machine learning	Méthodes qui apprennent des régularités à partir de données.	Prédire le risque de défaut d’un appareil.
Deep learning	Machine learning fondé sur des réseaux de neurones à plusieurs couches.	Reconnaissance vocale ou génération de texte.

Analogie : un apprenti

Un apprenti apprend à reconnaître des véhicules en observant de nombreux exemples. Un modèle d’apprentissage apprend lui aussi des régularités à partir d’exemples, mais au moyen de calculs et de paramètres numériques. Cette comparaison aide à comprendre l’apprentissage, sans signifier que la machine vit ou comprend comme un humain.

Ce qu’il faut retenir

- Une IA peut être performante dans une tâche étroite sans être capable de tout faire.
- Les performances dépendent des données, du modèle, du contexte et de la manière dont on mesure la réussite.
- Une IA générative produit du contenu : texte, images, audio, vidéo ou code.
- Un système d’IA peut être intégré dans un produit plus large contenant des règles, des outils et des bases de données.

Question de contrôle : un système qui recommande des films et un assistant qui écrit un e-mail sont-ils tous deux des systèmes d’IA ? Explique pourquoi.

2. Le réseau de neurones artificiels

Une intuition avant les équations

Un réseau de neurones artificiels est un ensemble d'unités de calcul organisées en couches. Les valeurs circulent entre ces unités, et les connexions possèdent des paramètres appelés poids. Pendant l'entraînement, ces paramètres sont ajustés pour améliorer un objectif.

Les couches

- Entrée — reçoit les données numériques, par exemple les pixels d'une image ou les valeurs d'un tableau.
- Couches cachées — transforment les données et apprennent des représentations utiles.
- Sortie — produit un résultat : classe, score, prédiction ou distribution de probabilités.

Les composants essentiels

Élément	Explication
Neurone artificiel	Unité qui combine des entrées, applique un calcul et transmet une valeur.
Poids	Paramètre qui règle l'influence d'une entrée.
Biais	Paramètre supplémentaire qui décale le calcul.
Fonction d'activation	Transformation non linéaire qui aide le réseau à représenter des relations complexes.
Fonction de perte	Mesure numérique de l'écart entre la sortie et l'objectif attendu.

Exemple

Un modèle reçoit une image d'un chiffre manuscrit. Il transforme les pixels en nombres, traite ces nombres dans plusieurs couches puis attribue des scores aux chiffres possibles. Si la bonne réponse est 7 mais que le modèle prédit 1, l'entraînement cherche à réduire cette erreur.

Analogie : accorder un instrument

Le musicien écoute un son, repère qu'il est trop aigu ou trop grave, puis ajuste une corde. Le réseau ajuste des paramètres à partir d'une mesure d'erreur. L'analogie illustre la boucle de correction, mais le réseau ne « ressent » pas son erreur.

Attention : le terme « neurone » est une inspiration biologique, pas la preuve qu'un réseau artificiel reproduit fidèlement le cerveau.

3. Comment un réseau apprend

Entraînement, rétropropagation et généralisation

Pendant l’entraînement, le modèle reçoit des exemples, produit des prédictions, mesure son erreur et ajuste ses paramètres. Cette boucle est répétée sur beaucoup d’exemples.

1. Le modèle reçoit un exemple d’entrée.
2. Il calcule une sortie : c’est la propagation vers l’avant.
3. Une fonction de perte compare la sortie à l’objectif attendu.
4. La rétropropagation calcule comment les paramètres ont contribué à l’erreur.
5. Un optimiseur, par exemple une variante de descente de gradient, ajuste les paramètres.
6. Le processus recommence sur d’autres exemples.

Trois notions qui reviennent souvent

- Rétropropagation — méthode de calcul des gradients de la fonction de perte par rapport aux paramètres.
- Descente de gradient — famille de méthodes qui modifient les paramètres dans une direction destinée à réduire la perte.
- Généralisation — capacité à bien traiter des données nouvelles, différentes de celles utilisées pour l’entraînement.

Surapprentissage et sous-apprentissage

Situation	Ce qui se passe	Réponse possible
Sous-apprentissage	Le modèle n’apprend pas suffisamment les régularités utiles.	Améliorer l’approche, les données ou la capacité du modèle.
Surapprentissage	Le modèle s’adapte trop aux exemples d’entraînement et généralise mal.	Plus de données variées, régularisation, validation correcte.
Bonne généralisation	Le modèle fonctionne aussi sur des exemples inédits.	Tester sur des données séparées et représentatives.

À ne pas confondre

L’entraînement ajuste les paramètres. L’inférence utilise un modèle déjà entraîné pour produire un résultat. Poser une question à un assistant ne réentraîne généralement pas son modèle de fond.

4. Les grands modèles de langage (LLM)

Tokens, contexte et génération

Un LLM (*Large Language Model*) est un modèle entraîné pour traiter et générer du langage. Les modèles modernes utilisent souvent une architecture Transformer ou une variante.

Comment une phrase est-elle générée ?

1. Le texte d'entrée est découpé en tokens.
2. Les tokens sont convertis en représentations numériques.
3. Le modèle traite le contexte avec ses paramètres et ses mécanismes internes.
4. Il calcule une distribution de probabilités sur les prochains tokens possibles.
5. Une stratégie de génération sélectionne un token, puis le processus recommence jusqu'à la fin de la réponse.

Le Transformer et l'attention

L'attention permet au modèle de pondérer l'importance de différents éléments du contexte pour certains calculs. Cela aide à traiter les relations entre des mots ou des éléments éloignés dans une séquence. Ce mécanisme ne garantit ni vérité ni raisonnement parfait.

Pourquoi le modèle peut-il se tromper ?

- Il peut produire une continuation plausible mais factuellement fausse.
- La demande peut être ambiguë ou contenir de fausses prémisses.
- Le contexte peut être trop long, incomplet ou contradictoire.
- Les informations récentes peuvent manquer si aucun outil de recherche n'est utilisé.
- Le modèle peut mal effectuer un calcul ou une chaîne de raisonnement.

Métaphore utile : le LLM est un moteur de langage très puissant. Pour devenir une application fiable, il peut avoir besoin de sources vérifiables, de règles, de tests et d'outils.

5. Les 50 mots essentiels — 1 à 10

Le vocabulaire de base

1. Intelligence artificielle (IA) — Domaine des méthodes qui permettent à des systèmes de réaliser des tâches telles que classer, prédire, générer ou planifier.

2. Machine learning — Apprentissage automatique : méthodes qui apprennent des régularités à partir de données.

3. Deep learning — Apprentissage automatique reposant sur des réseaux de neurones à plusieurs couches.

4. Réseau de neurones — Ensemble d’unités de calcul reliées, dont les paramètres sont ajustés pendant l’apprentissage.

5. Modèle — Système entraîné qui transforme une entrée en sortie selon des paramètres appris.

6. Paramètre — Valeur numérique apprise pendant l’entraînement, comme un poids ou un biais.

7. Entraînement — Processus d’ajustement des paramètres à partir de données et d’un objectif.

8. Inférence — Utilisation d’un modèle entraîné pour produire une prédiction ou une réponse.

9. Données d’entraînement — Exemples utilisés pour ajuster les paramètres du modèle.

10. Généralisation — Capacité à fonctionner sur des exemples nouveaux, pas seulement sur ceux déjà vus.

Mini-test

Explique en deux phrases la différence entre entraîner un modèle et utiliser un modèle. Donne un exemple de généralisation.

6. Les 50 mots essentiels — 11 à 20

Architecture et génération

11. LLM — Large Language Model : grand modèle de langage capable de traiter et générer du texte.

12. Transformer — Architecture de réseau de neurones largement utilisée pour le langage et d'autres données séquentielles.

13. Attention — Mécanisme qui pondère l'importance d'éléments du contexte dans les calculs du modèle.

14. Token — Unité de texte traitée par le modèle ; un token peut correspondre à un mot, un morceau de mot ou un signe.

15. Fenêtre de contexte — Quantité maximale de tokens que le modèle peut prendre en compte dans une requête donnée.

16. Embedding — Représentation numérique d'un élément, conçue pour capturer certaines relations ou similarités.

17. Prompt — Instruction et contexte fournis au modèle pour orienter sa réponse.

18. Prompt engineering — Conception et amélioration des prompts pour obtenir des résultats plus utiles et reproductibles.

19. Température — Paramètre de génération qui influence la diversité des choix de tokens ; son effet précis dépend du système.

20. Hallucination — Réponse générée qui semble plausible mais contient une information inventée, fausse ou non étayée.

À retenir

Un token n'est pas forcément un mot entier. La fenêtre de contexte n'est pas une mémoire permanente. Un prompt peut orienter une réponse, mais ne garantit pas qu'elle soit vraie.

7. Les 50 mots essentiels — 21 à 30

Applications et données

21. API — Interface qui définit comment un logiciel peut demander un service ou des données à un autre logiciel.

22. Backend — Partie serveur d’une application : logique métier, authentification, accès aux données et orchestration.

23. Frontend — Partie visible ou interactive avec laquelle l’utilisateur interagit.

24. Serveur — Ordinateur ou service qui reçoit des requêtes et fournit des réponses sur un réseau.

25. Base de données — Système organisé pour stocker, retrouver et modifier des informations.

26. RAG — Retrieval-Augmented Generation : méthode qui récupère des informations pertinentes puis les fournit au modèle pour répondre.

27. Recherche vectorielle — Recherche de contenus proches selon leurs représentations numériques, plutôt que seulement par mots exacts.

28. Fine-tuning — Adaptation supplémentaire d’un modèle préentraîné à l’aide d’exemples ou d’un objectif spécifique.

29. Multimodal — Capable de traiter ou produire plusieurs types de données : texte, image, audio ou vidéo.

30. Évaluation — Mesure systématique de la qualité d’un système selon des critères et des tests.

Question

Pourquoi une application peut-elle avoir besoin d’une base de données même si le modèle possède déjà des connaissances apprises ?

8. Les 50 mots essentiels — 31 à 40

Outils et agents

31. Agent IA — Système qui utilise un modèle pour choisir des étapes ou des actions afin d’atteindre un objectif, parfois avec des outils.

32. Outil (tool) — Fonction qu’un modèle ou un agent peut demander à l’application d’exécuter, par exemple une recherche ou un calcul.

33. Appel d’outil — Action structurée par laquelle le modèle indique qu’un outil doit être exécuté avec certains arguments.

34. MCP — Model Context Protocol : protocole standardisé permettant à des applications IA de se connecter à des outils et à des sources de contexte.

35. Serveur MCP — Programme qui expose des outils, des ressources ou des prompts selon le protocole MCP.

36. Client MCP — Application qui se connecte à un serveur MCP et découvre ou utilise ses capacités.

37. Skill — Ensemble de consignes, de connaissances et parfois de ressources réutilisables pour effectuer une tâche. Le sens précis dépend de la plateforme.

38. Workflow — Suite d’étapes organisées, souvent définies à l’avance, pour accomplir un processus.

39. Automatisation — Exécution d’une tâche par un système avec peu ou pas d’intervention humaine à chaque étape.

40. Orchestration — Coordination de modèles, outils, données et étapes dans une application.

Distinction importante

Un workflow suit souvent des étapes définies. Un agent peut choisir certaines étapes selon la situation. Un serveur MCP expose des capacités selon un protocole : il n’est pas en soi un agent intelligent.

9. Les 50 mots essentiels — 41 à 50

Fiabilité, sécurité et exploitation

41. Système de récupération — Composant qui retrouve des documents ou données pertinents à partir d’une requête.

42. Corpus — Ensemble de documents ou de données disponibles pour la recherche ou l’analyse.

43. Chunking — Découpage d’un document en morceaux plus petits, par exemple pour l’indexation ou le RAG.

44. Latence — Délai entre une demande et le résultat.

45. Débit — Quantité de requêtes ou de données traitées pendant une durée donnée.

46. Coût d’inférence — Coût lié à l’utilisation d’un modèle, souvent lié aux tokens, au matériel et au fournisseur.

47. Authentification — Vérification de l’identité d’un utilisateur ou d’un service.

48. Autorisation — Définition de ce qu’un utilisateur ou un service a le droit de faire.

49. Injection de prompt — Tentative de détourner le comportement du modèle en lui fournissant des instructions malveillantes dans le prompt ou les données.

50. Observabilité — Capacité à comprendre le fonctionnement d’un système à partir de journaux, métriques, traces et résultats.

Exercice de synthèse

Décris une application qui reçoit une question, recherche des documents, appelle un modèle, vérifie le résultat et l’affiche. Essaie d’utiliser au moins dix termes du glossaire.

10. Frontend, backend et serveur

Les pièces d’une application

Une application d’IA est rarement constituée d’un seul modèle. Elle est généralement composée de plusieurs parties qui se parlent.

Composant	Rôle	Analogie
Frontend	Affiche l’interface et reçoit les interactions.	Le comptoir où le client passe commande.
Backend	Applique la logique, vérifie les droits, prépare les requêtes et coordonne les services.	La cuisine et l’organisation du restaurant.
Serveur	Exécute un service et répond aux requêtes sur le réseau.	Le lieu où le travail est effectué.
Base de données	Stocke les comptes, paramètres, documents ou historiques nécessaires.	Le système d’archives.
Fournisseur de modèle	Héberge ou fournit l’accès au modèle d’IA.	Un spécialiste appelé pour une opération précise.

Exemple : assistant de support client

1. Le frontend affiche la boîte de dialogue.
2. Le backend reçoit le message et vérifie l’identité de l’utilisateur.
3. Le backend recherche les passages utiles dans la documentation.
4. Il envoie le message et les passages au modèle.
5. Il applique les règles nécessaires et renvoie la réponse.
6. Le frontend affiche la réponse et peut proposer une source ou une action suivante.

Pourquoi ne pas appeler le modèle directement depuis le navigateur ?

Dans de nombreuses applications, les clés secrètes, les permissions, la logique métier et les contrôles doivent rester côté serveur. Exposer une clé privée dans le code du navigateur permettrait à d’autres personnes de la récupérer. Il existe des exceptions et des architectures spécialisées, mais la règle générale est de protéger les secrets.

À retenir : le modèle est un composant ; l’application complète gère aussi les entrées, les accès, les outils, les erreurs et l’expérience utilisateur.

11. Comprendre une API

Comment deux logiciels communiquent

API signifie *Application Programming Interface*. Une API est un contrat qui décrit comment un logiciel peut demander une opération à un autre : quels points d’accès existent, quelles données envoyer, quel résultat attendre et quelles erreurs peuvent survenir.

Analogie : le menu d’un restaurant

Le client ne rentre pas en cuisine pour manipuler les ingrédients. Il choisit un élément du menu, passe commande, puis reçoit un résultat. Une API joue un rôle comparable : elle expose des opérations définies sans obliger l’appelant à connaître tous les détails internes.

Exemple de cycle API

1. Le logiciel prépare une requête avec une adresse, une méthode et des données.
2. Le service vérifie la requête et les droits.
3. Le service effectue l’opération demandée.
4. Il renvoie une réponse, souvent au format JSON.
5. Le logiciel appelant traite le résultat ou l’erreur.

Vocabulaire courant

- Endpoint — adresse précise à laquelle une opération est disponible.
- Méthode HTTP — indique le type d’action, par exemple GET pour lire ou POST pour soumettre des données.
- JSON — format textuel structuré couramment utilisé pour échanger des données.
- Clé API — secret ou identifiant utilisé pour authentifier certains appels ; elle doit être protégée.
- Quota et limite de débit — restrictions sur le volume d’appels ou la fréquence.
- Code de statut — indique si la requête a réussi ou rencontré un problème.

Mini-exemple conceptuel

Une application météo peut demander à une API les conditions actuelles pour une ville. L’API renvoie des données structurées, puis l’application les affiche sous forme de texte ou de graphique. L’API ne « pense » pas nécessairement : elle fournit un accès à un service.

Bon réflexe : lire la documentation de l’API, vérifier l’authentification, les limites, les erreurs et les conditions d’utilisation avant de l’intégrer.

12. Serveur, requête, réponse et erreurs

Le cycle de communication

Un serveur attend ou reçoit des requêtes et renvoie des réponses. Dans une application moderne, un même parcours peut traverser plusieurs serveurs : application, base de données, service de recherche et fournisseur de modèle.

Les étapes d’une requête

1. Le client envoie une requête réseau.
2. Le serveur reçoit et interprète les données.
3. Il vérifie les paramètres, l’identité et les autorisations.
4. Il exécute la logique demandée et appelle éventuellement d’autres services.
5. Il renvoie une réponse avec un résultat ou une erreur.

Erreurs fréquentes

Problème	Cause possible	Réflexe
401 Unauthorized	Identifiants absents ou invalides.	Vérifier l’authentification.
403 Forbidden	Identité reconnue mais action non autorisée.	Vérifier les permissions.
404 Not Found	Adresse ou ressource inexistante.	Vérifier l’endpoint et l’identifiant.
429 Too Many Requests	Limite de débit ou quota atteint.	Réduire la fréquence et respecter les délais de reprise.
5xx	Erreur côté service ou infrastructure.	Consulter les logs, réessayer prudemment et prévoir un mode dégradé.

Pourquoi les délais d’attente sont importants

Un service peut être lent ou indisponible. Une application robuste prévoit des délais d’attente, des reprises limitées, des messages d’erreur compréhensibles et une manière de ne pas répéter une action dangereuse. Réessayer sans limite peut amplifier une panne ou créer des opérations en double.

13. Comment choisir un modèle d’IA

Ne pas choisir seulement le modèle le plus célèbre

Il n’existe pas de meilleur modèle pour toutes les tâches. Un modèle peut exceller en programmation mais être trop coûteux pour une classification simple. Un autre peut être plus rapide, fonctionner localement ou mieux traiter certains formats.

Les critères de comparaison

Critère	Question à poser
Qualité	Réussit-il les tâches réelles qui m’intéressent ?
Raisonnement	Est-il fiable sur les problèmes à plusieurs étapes ?
Multimodalité	Accepte-t-il images, audio, vidéo ou documents selon mon besoin ?
Vitesse	La latence est-elle acceptable ?
Coût	Quel est le coût réel par tâche, y compris les outils et les reprises ?
Contexte	Peut-il recevoir la quantité de texte nécessaire ?
Confidentialité	Où les données sont-elles traitées et conservées ?
Déploiement	Est-il disponible par API, dans une application ou localement ?
Licence et limites	Quelles conditions s’appliquent à l’usage commercial ou à la redistribution ?

Familles de modèles

- Modèles généralistes — rédaction, synthèse, questions et tâches variées.
- Modèles de raisonnement — conçus pour consacrer davantage de calcul à certains problèmes complexes.
- Modèles de code — orientés vers la génération, l’explication ou la transformation de code.
- Modèles multimodaux — traitent plusieurs modalités, comme texte et image.
- Modèles spécialisés — optimisés pour un domaine ou une tâche particulière.
- Modèles locaux ou ouverts — peuvent offrir davantage de contrôle, au prix d’une gestion technique et matérielle.

Méthode de sélection

1. Définir trois à dix tâches représentatives.
2. Préparer des entrées identiques pour chaque modèle candidat.
3. Évaluer la qualité avec une grille claire.
4. Mesurer le temps de réponse, le coût et les erreurs.
5. Tester la confidentialité, la robustesse et les limites.
6. Choisir le modèle qui répond le mieux au besoin réel, pas celui qui impressionne dans une démonstration.

Règle : benchmarker sur son propre usage est plus utile que se fier à un classement général seul.

14. Température, tokens et fenêtre de contexte

Paramètres et contraintes pratiques

Les modèles de langage ne traitent pas toujours le texte comme des mots entiers. Ils utilisent des tokens, qui peuvent être des mots, des fragments de mots, des chiffres ou des signes. Le nombre de tokens varie selon la langue, le texte et le tokenizer.

La fenêtre de contexte

La fenêtre de contexte est la quantité de tokens que le modèle peut prendre en compte dans une requête donnée. Elle peut inclure les instructions, la conversation, les documents récupérés et la réponse en cours selon l'architecture et le produit.

Ce que la fenêtre n'est pas

- Ce n'est pas automatiquement une mémoire persistante entre deux conversations.
- Une grande fenêtre ne garantit pas que chaque détail sera utilisé correctement.
- Ajouter trop de documents peut augmenter le coût et diluer les informations pertinentes.
- Le contexte disponible dépend du modèle et de la configuration du produit.

La température

La température est un paramètre de génération qui modifie la distribution utilisée pour sélectionner les tokens. Une température plus basse tend souvent à rendre les sorties plus prévisibles ; une température plus élevée tend à augmenter la diversité. Les effets exacts dépendent de l'implémentation et ne remplacent pas l'évaluation.

Conseils pratiques

Tâche	Point de départ raisonnable
Extraction structurée	Favoriser la cohérence, imposer un schéma et valider le résultat.
Résumé factuel	Contexte pertinent, critères explicites et sources.
Idéation créative	Accepter davantage de variété et sélectionner les meilleures propositions.
Code critique	Tests automatiques, revue humaine et exécution en environnement contrôlé.

15. Embeddings et recherche vectorielle

Retrouver des contenus par proximité sémantique

Un embedding est une représentation numérique d’un texte, d’une image ou d’un autre élément. Dans de nombreux systèmes, des contenus ayant des significations proches produisent des vecteurs proches selon une mesure de similarité choisie.

Pourquoi c’est utile

Une recherche par mots exacts peut manquer une question formulée différemment du document. Une recherche vectorielle peut retrouver un passage proche par le sens, même si les mots ne sont pas identiques. Elle n’est toutefois pas infaillible : les termes exacts, les filtres et la recherche hybride peuvent rester importants.

Pipeline simplifié

1. Rassembler les documents autorisés.
2. Nettoyer et découper le contenu en morceaux raisonnables.
3. Calculer un embedding pour chaque morceau.
4. Stocker les vecteurs et les métadonnées dans un index.
5. Transformer la question en vecteur.
6. Retrouver les morceaux les plus pertinents et, si nécessaire, les réordonner.
7. Fournir les passages sélectionnés au modèle avec des instructions claires.

Notions associées

- Similarité cosinus — mesure courante de proximité entre vecteurs.
- Métadonnées — informations supplémentaires, comme la date, la source, la catégorie ou les droits d’accès.
- Recherche hybride — combine recherche lexicale et recherche sémantique.
- Reranking — réévaluation et reclassement des résultats récupérés.
- Index — structure optimisée pour retrouver efficacement des éléments.

Attention : une proximité vectorielle n’est pas une preuve que le passage répond à la question. Il faut évaluer la pertinence et vérifier les droits d’accès.

16. RAG : connecter un modèle à ses documents

Retrieval-Augmented Generation

Le RAG combine la récupération d'informations externes et la génération de texte. Au lieu de demander au modèle de répondre uniquement à partir de ses paramètres, l'application recherche des passages pertinents et les ajoute au contexte.

Architecture typique

1. Préparer les documents : extraction du texte, nettoyage et découpage.
2. Indexer les passages : embeddings, index lexical ou approche hybride.
3. Recevoir une question et rechercher les passages pertinents.
4. Appliquer les filtres d'accès et éventuellement reclasser les résultats.
5. Envoyer la question, les passages et les règles au modèle.
6. Générer une réponse avec références aux sources quand c'est possible.
7. Évaluer la réponse et les passages utilisés.

Ce que le RAG améliore

- Permet d'utiliser des documents internes ou actualisés sans réentraîner le modèle.
- Peut rendre les réponses plus vérifiables si les sources sont correctement conservées.
- Peut séparer les connaissances générales du modèle des données propres à une organisation.
- Facilite la mise à jour des documents, sous réserve d'une indexation correcte.

Ce que le RAG ne garantit pas

- Il ne garantit pas que la bonne source sera retrouvée.
- Il ne garantit pas que le modèle respectera fidèlement les passages.
- Il ne corrige pas automatiquement des documents faux ou périmés.
- Il ne remplace pas les permissions, les contrôles de confidentialité ou les tests.

Quand choisir le RAG ?

Il est souvent pertinent quand les réponses doivent s'appuyer sur une documentation qui change, un corpus interne ou des sources traçables. Pour un simple assistant de rédaction générale, il peut être inutile. Commence par vérifier que le besoin existe avant d'ajouter une architecture complexe.

17. Base de données, mémoire et historique

Trois notions différentes

Le mot « mémoire » est souvent utilisé de manière imprécise dans les applications IA. Il faut distinguer ce que le modèle a appris pendant son entraînement, le contexte actuel, l'historique conservé par l'application et une base documentaire consultée à la demande.

Mécanisme	Ce qu'il contient	Durée typique
Paramètres du modèle	Régularités apprises pendant l'entraînement.	Jusqu'à une nouvelle version ou adaptation.
Contexte de la requête	Messages et informations fournis au modèle maintenant.	Limité à la requête ou à la session selon le système.
Historique de conversation	Messages conservés par l'application.	Selon les réglages et politiques du produit.
Mémoire applicative	Informations sélectionnées pour personnaliser les interactions.	Selon le système et les contrôles disponibles.
Base documentaire / RAG	Documents retrouvés à la demande.	Selon la conservation et l'indexation des documents.

Exemple

Si un assistant doit connaître les horaires actuels d'un service, il vaut mieux consulter une source à jour qu'espérer que le modèle s'en souvienne. Si l'application doit retrouver un contrat précis, il faut une recherche documentaire. Si elle doit conserver les préférences d'un utilisateur, il faut une fonctionnalité de mémoire conçue à cette fin.

Question à se poser

L'information doit-elle être apprise dans le modèle, transmise dans le contexte, enregistrée dans une base ou récupérée depuis une source externe ? La bonne réponse dépend du besoin, des mises à jour et des règles de confidentialité.

18. MCP : comprendre le protocole

Model Context Protocol

MCP signifie Model Context Protocol. C’est un protocole qui standardise la manière dont une application compatible peut découvrir et utiliser des capacités exposées par des serveurs : outils, ressources et prompts selon l’implémentation.

L’idée centrale

Sans standard commun, chaque application peut nécessiter une intégration particulière pour chaque outil. Un protocole partagé réduit le besoin de créer une connexion totalement différente pour chaque combinaison, même si les autorisations et les détails de déploiement doivent toujours être gérés.

Les rôles principaux

Élément	Rôle
Host	Application dans laquelle l’utilisateur travaille et qui gère l’expérience globale.
Client MCP	Composant qui maintient la connexion et échange avec un serveur MCP.
Serveur MCP	Expose des outils, ressources ou prompts via le protocole.
Outil	Action appelable, avec une description et des paramètres structurés.
Ressource	Contenu ou donnée que le client peut lire selon les capacités exposées.
Prompt	Modèle de message ou instruction réutilisable éventuellement proposé par le serveur.

Analogie

Imagine un adaptateur standard pour connecter plusieurs accessoires. MCP joue un rôle de standard de communication, pas de cerveau. Il ne rend pas un outil intelligent à lui seul ; il facilite la manière dont une application découvre et appelle cet outil.

MCP n’est pas…

- un modèle d’IA ;
- une base de données à lui seul ;
- une garantie de sécurité ;
- un agent autonome ;
- une preuve que chaque serveur est fiable.

Référence officielle : https://modelcontextprotocol.io/introduction

19. Comment fonctionne un serveur MCP ?

De la découverte d’un outil à son exécution

Un serveur MCP est un programme qui annonce les capacités qu’il expose et traite les demandes compatibles. Selon sa configuration, il peut s’exécuter localement ou être accessible à distance. Les transports et détails de déploiement dépendent de la version et de l’implémentation.

Cycle simplifié

1. L’application hôte se connecte au serveur MCP.
2. Le client et le serveur négocient ou vérifient les capacités et la version prises en charge.
3. Le client découvre les outils disponibles et leurs descriptions.
4. Le modèle ou l’orchestrateur sélectionne éventuellement un outil approprié.
5. L’application valide l’appel et le transmet au serveur.
6. Le serveur exécute l’action avec les permissions dont il dispose.
7. Le résultat est renvoyé à l’application, qui peut le fournir au modèle pour la suite.

Exemple : serveur MCP de fichiers

Un serveur peut exposer des opérations permettant de lister certains fichiers ou de lire leur contenu. Le fait qu’un outil existe ne signifie pas qu’il doit avoir accès à tout le disque. Il faut limiter les répertoires, les permissions, les types d’opérations et les données renvoyées.

Exemple : serveur MCP pour une base de données

Un serveur peut proposer une recherche prédéfinie ou des opérations de lecture. Il est généralement plus prudent de fournir des outils étroits et contrôlés que d’exposer un accès illimité à la base ou de laisser des requêtes arbitraires modifier des données.

Bonnes pratiques

- N’installer que des serveurs provenant de sources que l’on peut évaluer.
- Vérifier le code, la documentation, les permissions et les destinations réseau lorsque c’est possible.
- Limiter les droits au strict nécessaire.
- Demander une confirmation avant les actions sensibles ou irréversibles.
- Ne pas transmettre de secrets ou de données confidentielles sans nécessité.
- Journaliser les actions utiles sans conserver inutilement des données sensibles.

Le protocole standardise la communication ; la sécurité dépend aussi du serveur, de sa configuration, de ses permissions et de l’application qui l’utilise.

20. Skills, outils et workflows

Ne pas confondre les briques

Le mot « skill » peut désigner des choses différentes selon la plateforme. Dans ce guide, il désigne un ensemble réutilisable de consignes, de procédures et éventuellement de fichiers de référence qui aide à effectuer une catégorie de tâche.

Brique	Question à laquelle elle répond	Exemple
Prompt	Que dois-je faire maintenant ?	Résume ce texte en cinq points.
Skill	Quelle méthode réutilisable appliquer ?	Procédure de revue d'un contrat.
Outil	Quelle action concrète puis-je exécuter ?	Chercher un document ou calculer un total.
API	Comment demander cette action à un service ?	Envoyer une requête à un service de recherche.
Serveur MCP	Comment exposer des outils ou ressources selon MCP ?	Fournir un outil de recherche à une application compatible.
Workflow	Quelles étapes suivre dans quel ordre ?	Lire un document, extraire les clauses, produire un rapport.
Agent	Comment choisir les prochaines étapes pour atteindre l'objectif ?	Planifier une recherche et appeler plusieurs outils.

Exemple de skill bien conçu

Nom : « Préparer une réunion ». But : produire une synthèse exploitable. Entrées : notes, participants, objectif et durée. Étapes : identifier les décisions attendues, regrouper les sujets, distinguer faits et hypothèses, proposer un ordre du jour, signaler les informations manquantes. Sortie : ordre du jour, questions ouvertes et actions. Contrôle : ne pas inventer de décisions ou d'engagements.

Une skill n'est pas forcément un programme. Elle peut être essentiellement documentaire. Un outil, en revanche, exécute une opération concrète. Les définitions précises dépendent du produit utilisé.

21. Agent IA ou automatisation classique ?

Quand faut-il donner de l’autonomie ?

Un agent IA combine généralement un modèle, un objectif, un contexte, des outils et une boucle de décision. Il peut choisir certaines actions en fonction des résultats intermédiaires. Une automatisation classique suit plutôt des règles et des étapes préétablies.

Approche	Avantages	Risques / limites
Règles classiques	Prévisible, facile à tester, souvent peu coûteuse.	Moins flexible face aux cas inattendus.
Workflow avec IA	Combine étapes fixes et compréhension du langage.	Les sorties du modèle doivent être contrôlées.
Agent avec outils	Peut adapter les étapes à la situation.	Plus difficile à tester ; risque d’actions inadaptées et de coûts imprévisibles.

Exemple : traitement de factures

Un workflow robuste peut extraire les champs, vérifier les totaux, appliquer des règles déterministes et envoyer les cas incertains à une personne. Un agent libre qui décide seul de payer ou de modifier les comptes serait beaucoup plus risqué. L’autonomie doit être proportionnelle aux conséquences d’une erreur.

Règle de conception

Commencer par une automatisation déterministe. Ajouter l’IA là où l’interprétation est utile. Ajouter des choix autonomes seulement si le système est mesurable, contrôlable et réversible.

22. Construire un bon prompt

Une méthode reproductible

Un prompt utile ne se résume pas à une formule magique. Il réduit l'ambiguïté en précisant l'objectif, le contexte, les données disponibles, les contraintes et la forme attendue.

Les six blocs d'un prompt

Bloc	Ce qu'il précise	Exemple
Rôle / perspective	Le point de vue utile, sans prétendre changer les capacités réelles.	Agis comme un formateur débutant.
Objectif	Le résultat à produire.	Explique le fonctionnement d'une API.
Contexte	Le niveau, le public et la situation.	Le lecteur ne sait pas programmer.
Entrées	Le texte ou les données à traiter.	Voici le descriptif du projet…
Contraintes	Longueur, ton, exclusions, exigences.	Évite le jargon ou définis-le.
Format	Structure de la sortie.	Tableau, étapes et exemple concret.

Modèle de prompt réutilisable

« Objectif : [résultat]. Contexte : [situation et niveau]. Entrées : [données disponibles]. Contraintes : [limites, ton, longueur, sources]. Procédure : [étapes souhaitées]. Format de sortie : [structure]. Si une information indispensable manque, pose une question au lieu de l'inventer. Signale les incertitudes et distingue les faits des hypothèses. »

Avant / après

Prompt faible : « Explique-moi l'IA. »

Prompt amélioré : « Explique les bases de l'IA à un adulte débutant. Utilise une analogie simple, distingue IA, machine learning et deep learning, donne deux exemples et termine par trois questions de vérification. Définis chaque terme technique. Ne présente pas les analogies comme des équivalences exactes. »

Un bon prompt aide, mais ne garantit pas le résultat. Il faut examiner la réponse, demander des corrections ciblées et vérifier les faits importants.

23. Techniques de prompting utiles

Exemples, étapes, critères et formats

Techniques courantes

- Décomposer — demander un plan ou des étapes quand la tâche est complexe.
- Donner un exemple — fournir un exemple de format ou de niveau de qualité attendu.
- Définir des critères — dire ce qui compte : exactitude, clarté, coût, concision ou traçabilité.
- Imposer un format — demander une table, un JSON validable ou une liste structurée.
- Séparer données et instructions — utiliser des délimiteurs clairs autour du contenu fourni.
- Demander les incertitudes — faire expliciter les informations manquantes et les hypothèses.
- Itérer — corriger un aspect à la fois plutôt que tout réécrire sans diagnostic.
- Fournir des sources — demander de s’appuyer sur des documents identifiables quand le sujet l’exige.

Exemple de tâche structurée

« À partir du texte ci-dessous, extrais : nom du fournisseur, date, montant, échéance et clauses à vérifier. Si un champ n’est pas présent, écris “non trouvé”. N’infère pas les données manquantes. Renvoie un tableau puis une liste de points à vérifier. Texte : [insérer le texte]. »

Quand demander une question de clarification ?

Quand une information manquante change fortement le résultat : pays concerné, public cible, budget, format de livraison, date, niveau de risque ou définition de la réussite. Pour une tâche simple, il est souvent plus efficace de demander une seule précision essentielle plutôt que d’imposer un long questionnaire.

À éviter

- Des instructions contradictoires (« très court » et « exhaustif » sans hiérarchie).
- Des demandes vagues comme « fais mieux » sans critère précis.
- Des exigences impossibles à vérifier, comme « sois parfaitement exact » sans source.
- Des prompts gigantesques qui mélangent plusieurs objectifs sans structure.
- La confiance aveugle dans une sortie simplement parce qu’elle est bien rédigée.

24. Vérifier les réponses et limiter les hallucinations

La fluidité n’est pas la vérité

Une hallucination est une sortie qui présente une information fausse, inventée ou insuffisamment étayée comme si elle était fiable. Le terme est particulièrement utilisé pour les modèles génératifs.

Méthode de vérification en cinq étapes

1. Identifier les affirmations importantes et vérifiables.
2. Repérer les affirmations chiffrées, juridiques, médicales, financières ou récentes.
3. Comparer les affirmations aux sources primaires ou à des références fiables.
4. Vérifier les calculs, unités, dates et conditions.
5. Corriger ou supprimer les éléments qui ne sont pas suffisamment étayés.

Demander une réponse plus vérifiable

« Sépare les faits directement présents dans les documents, les déductions et les points incertains. Pour chaque affirmation importante, indique la source précise si elle est disponible. Si les documents ne permettent pas de conclure, écris-le clairement. »

Les sources aussi peuvent être mauvaises

Un lien peut être périmé, une page peut être incorrecte, et un document fourni peut contenir une erreur. La vérification ne consiste pas seulement à trouver une source, mais à évaluer son autorité, sa date, sa pertinence et ce qu’elle prouve réellement.

Niveaux de risque

Risque faible	Risque moyen	Risque élevé
Idées de titres, brouillon créatif.	Analyse interne, résumé, recommandation opérationnelle.	Santé, droit, finances, sécurité, recrutement ou action irréversible.

Plus l’impact d’une erreur est important, plus il faut des sources solides, des contrôles déterministes et une validation humaine compétente.

25. Confidentialité, sécurité et injection de prompt

Ne pas donner plus de pouvoir que nécessaire

Une application d’IA peut traiter des données personnelles, des documents internes, des secrets commerciaux ou des identifiants. Avant d’envoyer des informations, il faut comprendre où elles sont traitées, qui peut y accéder, combien de temps elles sont conservées et à quelles fins elles peuvent être utilisées.

Principes de base

- Ne jamais coller une clé API, un mot de passe ou un secret dans un prompt public.
- Minimiser les données envoyées : ne transmettre que ce qui est nécessaire.
- Vérifier les paramètres de confidentialité et les conditions du fournisseur.
- Limiter les permissions des outils et des serveurs.
- Prévoir des confirmations avant les actions sensibles.
- Séparer les environnements de test et de production.
- Conserver des journaux utiles sans stocker inutilement des données privées.

Injection de prompt

Une injection de prompt est une tentative de détourner le comportement du modèle au moyen d’instructions malveillantes placées dans une demande ou dans un contenu récupéré. Par exemple, un document peut contenir une instruction disant d’ignorer les règles et d’exfiltrer des informations. Le contenu documentaire doit être traité comme une donnée non fiable, pas comme une autorité supérieure.

Défenses en couches

1. Ne pas compter uniquement sur le prompt système pour protéger les secrets.
2. Appliquer les permissions dans le code et côté serveur.
3. Restreindre les outils et leurs paramètres.
4. Valider les sorties avant toute action importante.
5. Demander une approbation humaine pour les opérations risquées.
6. Tester les attaques et les cas limites avant le déploiement.

Principe fondamental : un modèle peut suggérer une action, mais les règles d’autorisation doivent être appliquées par le système qui exécute l’action.

26. Construire un premier assistant IA

Une architecture simple avant de viser l’autonomie

Projet : assistant qui répond à des questions à partir d’un petit guide de procédures. Le but est de construire un système utile, mesurable et limité.

Étape 1 — Définir le besoin

Exemple : « aider les nouveaux employés à trouver les procédures internes, avec une référence au document utilisé ». Ne commence pas par choisir un modèle. Commence par définir le problème et ce qu’une bonne réponse signifie.

Étape 2 — Préparer les documents

- Choisir des documents à jour et autorisés.
- Supprimer les doublons et les versions obsolètes.
- Identifier la source, la date et le propriétaire de chaque document.
- Définir quels utilisateurs peuvent consulter chaque document.

Étape 3 — Créer la chaîne de traitement

1. Recevoir la question.
2. Rechercher les passages pertinents.
3. Vérifier que les passages sont accessibles à l’utilisateur.
4. Construire le contexte et les instructions.
5. Appeler le modèle.
6. Afficher une réponse avec sources et incertitudes.
7. Proposer une escalade vers une personne si les sources ne suffisent pas.

Étape 4 — Définir les limites

L’assistant doit pouvoir dire « je ne trouve pas l’information ». Il ne doit pas inventer une procédure, contourner des permissions ou prendre des décisions RH ou financières sans validation appropriée.

Étape 5 — Mesurer

Prépare un jeu de 30 à 50 questions représentatives. Note si le bon document est retrouvé, si la réponse est correcte, si les sources la soutiennent, si le système refuse correctement les demandes hors périmètre et si le délai reste acceptable.

27. Évaluer une application IA

Passer de l’impression aux mesures

Une démo réussie ne prouve pas qu’un système est prêt pour un usage réel. L’évaluation doit utiliser des cas représentatifs, des critères explicites et des tests reproductibles.

Grille d’évaluation simple

Critère	Question	Exemple de mesure
Exactitude	La réponse est-elle correcte ?	Part des réponses acceptées par des évaluateurs.
Pertinence	Répond-elle à la demande ?	Note sur une échelle définie.
Traçabilité	Les sources soutiennent-elles les affirmations ?	Taux d’affirmations correctement référencées.
Robustesse	Que se passe-t-il avec des entrées ambiguës ?	Taux d’échec sur cas limites.
Sécurité	Le système protège-t-il les données et permissions ?	Résultats de tests d’accès et d’injection.
Latence	Le résultat arrive-t-il assez vite ?	Médiane et percentiles de délai.
Coût	Quel est le coût par tâche réussie ?	Coût total divisé par tâches valides.

Créer un jeu de test

1. Collecter des questions réellement posées ou plausibles.
2. Écrire les réponses attendues ou les critères d’acceptation.
3. Inclure des cas simples, difficiles, ambigus et hors périmètre.
4. Conserver le jeu de test séparément des exemples utilisés pour ajuster le système.
5. Relancer les tests après chaque changement de prompt, modèle ou index documentaire.
6. Examiner les erreurs, puis corriger la cause plutôt que masquer le symptôme.

L’évaluation doit couvrir le système entier : récupération, permissions, modèle, format de sortie, outils et interface.

28. Coût, vitesse et choix d’architecture

Le plus sophistiqué n’est pas toujours le meilleur

Le coût d’une application IA ne se limite pas au prix d’un appel au modèle. Il peut inclure l’hébergement, la recherche, les embeddings, le stockage, les outils externes, les logs, les reprises et le temps de maintenance.

Leviers d’optimisation

- Utiliser un petit modèle pour les tâches simples et un modèle plus puissant pour les cas difficiles.
- Réduire le contexte inutile et récupérer des passages ciblés.
- Mettre en cache les résultats qui peuvent être réutilisés sans risque.
- Éviter les appels redondants et les boucles d’agent sans limite.
- Mesurer les coûts par tâche réussie, pas seulement par requête.
- Prévoir des plafonds, des délais d’attente et des limites d’utilisation.

Choisir l’architecture

Option	Pertinente si…	Point de vigilance
Appel direct à un modèle	La tâche est simple et le contexte est fourni.	Pas de recherche ni de mémoire persistante par défaut.
Modèle + RAG	Les réponses doivent s’appuyer sur des documents externes.	Qualité de l’index et des sources.
Modèle + outils	Des données ou actions externes sont nécessaires.	Permissions, validation et erreurs d’outils.
Agent multi-étapes	Le plan doit s’adapter aux résultats.	Coût, imprévisibilité et difficulté d’évaluation.

Commence avec l’architecture la plus simple qui satisfait les critères de qualité. Ajoute une nouvelle brique seulement si elle résout un problème identifié.

29. Modèles ouverts, modèles propriétaires et exécution locale

Contrôle, coût et compromis

Les modèles propriétaires sont fournis sous des conditions définies par un fournisseur, souvent via une application ou une API. Les modèles à poids ouverts rendent leurs paramètres accessibles sous certaines conditions de licence. « Poids ouverts » ne signifie pas automatiquement que toutes les données d'entraînement, le code ou les droits sont ouverts.

Dimension	API d'un fournisseur	Modèle exécuté localement
Mise en route	Souvent simple.	Nécessite installation et configuration.
Infrastructure	Gérée en grande partie par le fournisseur.	Gérée par l'utilisateur ou son organisation.
Confidentialité	Dépend des conditions et du traitement du fournisseur.	Peut améliorer le contrôle, mais dépend de la configuration locale.
Qualité	Accès à des modèles très performants selon l'offre.	Dépend du modèle, du matériel et de la tâche.
Coût	Facturation selon l'usage ou le forfait.	Matériel, énergie, maintenance et temps d'administration.
Contrôle	Dépend de l'API et des options disponibles.	Plus de contrôle sur l'environnement, mais plus de responsabilités.

Comment décider ?

Définis d'abord les exigences : sensibilité des données, niveau de qualité, volume d'usage, latence, budget, matériel disponible, licence et compétences techniques. Compare ensuite des solutions concrètes sur les mêmes tests. Il n'existe pas de réponse universelle.

30. IA multimodale

Texte, image, audio et vidéo

Un système multimodal peut traiter plusieurs types de données. Certains modèles acceptent des images et du texte, d'autres traitent l'audio, et certains produits combinent plusieurs modalités dans une même interaction.

Exemples

- Vision — décrire une image, extraire du texte d'un document photographié, comparer des éléments visuels.
- Audio — transcrire la parole, analyser un enregistrement ou générer une voix.
- Vidéo — résumer des scènes ou analyser des images extraites à certains instants.
- Documents — combiner texte extrait, mise en page et images selon les capacités du système.
- Voix en temps réel — combiner entrée audio, traitement et réponse vocale avec des contraintes de latence.

Limites pratiques

Une image basse résolution, un son bruité, un accent, un document mal numérisé ou une vidéo trop longue peut réduire la qualité. L'application doit aussi respecter les droits d'accès et les règles de confidentialité des médias.

Bon réflexe

Pour une tâche multimodale, précise ce que le système doit observer, ce qu'il doit ignorer, le format de sortie et comment signaler l'incertitude. Vérifie les détails critiques avec une source ou un outil spécialisé si nécessaire.

31. Les limites fondamentales de l’IA

Ce qu’il ne faut pas supposer

Les modèles peuvent produire des résultats impressionnants sans être fiables sur chaque cas. Leur performance dépend de la tâche, des données, de la conception, des outils et des contrôles.

Limites fréquentes

- Les connaissances peuvent être incomplètes ou datées.
- Les réponses peuvent contenir des erreurs de raisonnement ou de calcul.
- Les performances peuvent varier selon la langue, le domaine ou le type d’entrée.
- Les modèles peuvent reproduire des biais présents dans les données.
- Un contexte long ne garantit pas une utilisation parfaite de chaque détail.
- Les résultats peuvent varier d’un appel à l’autre.
- Les outils peuvent échouer, renvoyer des données incomplètes ou changer de comportement.
- Une réponse plausible ne démontre pas une compréhension humaine ou une conscience.

L’analogie doit rester une analogie

Dire « un réseau de neurones ressemble vaguement à un cerveau » peut aider à visualiser l’idée de connexions et d’apprentissage. Mais il serait incorrect d’en conclure que le modèle a les mêmes mécanismes biologiques, la même expérience subjective ou la même manière de comprendre que l’humain.

Approche professionnelle

Associer l’IA à des règles explicites, des sources, des tests, des permissions et une supervision proportionnée au risque. L’IA peut être une excellente aide sans être l’autorité finale.

32. Les erreurs de débutant à éviter

Les pièges les plus fréquents

- Choisir un outil avant le problème — définis d’abord le résultat recherché et les contraintes.
- Croire une réponse bien écrite — vérifie les faits importants et les sources.
- Confondre modèle et application — le modèle n’est qu’une brique dans de nombreux produits.
- Confondre mémoire et contexte — la fenêtre de contexte ne constitue pas automatiquement une mémoire permanente.
- Ajouter un agent partout — un workflow simple est souvent plus fiable.
- Donner trop de permissions — limite les outils et les accès au strict nécessaire.
- Utiliser un seul exemple pour évaluer — teste une variété de cas réels et difficiles.
- Ignorer les coûts — mesure les appels, la latence, les reprises et le coût de maintenance.
- Envoyer des données sensibles sans vérifier — comprends les règles de traitement et minimise les données.
- Automatiser une action irréversible — ajoute une confirmation et un mécanisme de retour arrière.

Check-list avant de déployer

Question	Oui / Non
Le problème et le résultat attendu sont-ils clairement définis ?	■
Les données sont-elles autorisées et à jour ?	■
Les sorties ont-elles été testées sur des cas représentatifs ?	■
Les permissions sont-elles minimales ?	■
Les erreurs et pannes ont-elles un comportement prévu ?	■
Le coût et la latence sont-ils mesurés ?	■
Une personne peut-elle reprendre la main sur les cas sensibles ?	■

33. Exercice 1 — Expliquer un concept simplement

Apprendre à reformuler sans déformer

Objectif : vérifier que tu comprends un concept suffisamment bien pour l'expliquer à quelqu'un d'autre.

Consigne

Choisis l'un des concepts suivants : réseau de neurones, API, RAG, MCP, token ou agent IA. Rédige une explication de 100 à 150 mots pour une personne qui ne connaît aucun terme technique.

Contraintes

- Une définition simple en une phrase.
- Une analogie concrète.
- Un exemple réel.
- Une limite de l'analogie ou du concept.
- Une phrase de synthèse commençant par « À retenir ».

Grille d'auto-évaluation

Critère	0 point	1 point	2 points
Exactitude	Concept faux	Partiellement correct	Correct et nuancé
Clarté	Jargon non expliqué	Quelques passages obscurs	Accessible au débutant
Exemple	Absent	Générique	Concret et pertinent
Limite	Aucune	Mentionnée vaguement	Limite clairement expliquée

Réussite : obtenir au moins 6/8 points, puis demander à une autre personne de reformuler le concept sans regarder ton texte.

34. Exercice 2 — Améliorer un prompt

Passer d’une demande vague à une consigne exploitable

Prompt initial : « Fais-moi un plan marketing. »

Travail demandé

1. Liste les informations manquantes : produit, cible, marché, budget, délai, objectif et canal.
2. Décide quelles informations sont indispensables pour produire une première version.
3. Rédige un prompt qui demande une stratégie, des hypothèses explicites, un calendrier et des indicateurs.
4. Ajoute une consigne pour ne pas inventer des chiffres de marché.
5. Évalue la réponse selon sa pertinence, sa faisabilité et ses hypothèses.

Exemple de version améliorée

« Construis un plan marketing de 90 jours pour [produit], destiné à [public], avec un budget de [montant]. Objectif principal : [objectif mesurable]. Propose trois priorités, un calendrier hebdomadaire, les canaux, les ressources nécessaires et les indicateurs. Sépare les données fournies des hypothèses. N’invente pas de statistiques de marché : indique celles qu’il faudrait rechercher. Si une information essentielle manque, pose jusqu’à trois questions avant de finaliser. »

Extension

Demande ensuite une critique du plan : « Identifie les trois hypothèses les plus fragiles, les coûts sous-estimés et les raisons pour lesquelles ce plan pourrait échouer. Propose un test peu coûteux pour chaque risque. »

35. Exercice 3 — Dessiner une application IA

Du besoin au schéma d’architecture

Scénario : une petite association veut un assistant qui répond aux questions à partir de ses documents et propose un contact humain quand l’information manque.

Dessine les blocs suivants

- Interface de conversation (frontend).
- Backend qui reçoit la demande et vérifie l’identité.
- Système de recherche documentaire.
- Documents et index.
- Modèle de langage.
- Contrôle des sources et format de réponse.
- Escalade vers un bénévole.
- Journalisation minimale et mesure de qualité.

Questions à résoudre

1. Où les documents sont-ils stockés et qui peut y accéder ?
2. Que se passe-t-il si aucun document pertinent n’est retrouvé ?
3. Comment l’utilisateur peut-il vérifier la source de la réponse ?
4. Comment signaler une réponse incorrecte ?
5. Quels types de questions doivent être transmis à un humain ?
6. Quels tests démontrent que l’assistant fonctionne correctement ?

Livrable attendu : un schéma simple, une liste de risques et cinq tests d’acceptation.

36. Exercice 4 — Concevoir un outil et son serveur MCP

Exposer une capacité sans ouvrir tous les accès

Scénario : un assistant doit pouvoir rechercher les horaires d’ouverture d’une organisation. Il n’a pas besoin de modifier les horaires.

Conception de l’outil

Élément	Décision à prendre
Nom	search_opening_hours
Description	Recherche les horaires publiés pour un lieu ou un service.
Entrées	Identifiant du lieu ou nom contrôlé.
Sortie	Horaires, source, date de mise à jour si disponible.
Permissions	Lecture seule.
Erreurs	Lieu inconnu, source indisponible, données ambiguës.
Limites	Pas de modification, pas d’accès à des données privées.

Questions de sécurité

- Le serveur peut-il accéder uniquement aux données nécessaires ?
- Les entrées sont-elles validées ?
- La sortie indique-t-elle sa source et sa date si possible ?
- Le système distingue-t-il les données manquantes des horaires fermés ?
- Les erreurs sont-elles explicites sans divulguer de secrets ?
- Le client demande-t-il une confirmation pour les actions à risque ?

Leçon : un bon outil a un objectif étroit, une description claire, des entrées contrôlées et des permissions minimales. Exposer une capacité ne signifie pas donner un accès illimité.

37. Mini-projet — Assistant documentaire

Un projet réaliste pour consolider les acquis

But : créer un assistant qui répond à partir d’un corpus limité de documents et cite les passages utilisés.

Version 1 — Sans RAG

Choisis un seul document court, fournis-le dans le contexte et demande au modèle de répondre uniquement à partir de ce contenu. Ajoute une règle de refus quand la réponse n’est pas présente.

Version 2 — Recherche documentaire

Quand le corpus grandit, découpe les documents, indexe les passages, recherche les plus pertinents et construis le contexte. Affiche le nom du document et, si possible, la page ou la section.

Version 3 — Évaluation

1. Préparer 30 questions auxquelles les documents répondent.
2. Préparer 10 questions dont la réponse n’est pas présente.
3. Préparer 5 questions ambiguës.
4. Mesurer si les bons passages sont retrouvés.
5. Vérifier que les réponses sont fidèles aux sources.
6. Tester les demandes d’accès interdit.
7. Documenter les erreurs et les corrections.

Critères de réussite

- Le système ne présente pas comme un fait une information absente des documents.
- Les sources affichées correspondent réellement à la réponse.
- Les utilisateurs ne peuvent pas accéder aux documents non autorisés.
- Le temps de réponse et le coût restent acceptables.
- Les erreurs peuvent être signalées et reproduites.

N’ajoute un serveur MCP ou un agent que si le projet en a réellement besoin. La première version doit rester simple.

38. Plan d’apprentissage sur 30 jours

Un programme court, concret et progressif

Période	Objectif	Travail
Jours 1–3	Comprendre les bases	Lire les chapitres IA, réseaux de neurones, entraînement et LLM ; expliquer chaque notion sans notes.
Jours 4–6	Maîtriser le vocabulaire	Apprendre les 50 termes ; créer des cartes de révision.
Jours 7–9	Prompting	Écrire dix prompts, comparer les réponses et améliorer les consignes.
Jours 10–12	Architecture web	Dessiner frontend, backend, serveur, API et base de données.
Jours 13–15	Données et RAG	Créer un petit corpus, découper les documents et réfléchir à la recherche.
Jours 16–18	MCP et outils	Étudier un exemple de serveur MCP, les entrées, sorties et permissions.
Jours 19–21	Sécurité	Lister les données sensibles, les droits et les actions à confirmer.
Jours 22–25	Mini-projet	Construire un assistant documentaire simple.
Jours 26–28	Évaluation	Créer des tests, mesurer les erreurs et améliorer le système.
Jours 29–30	Synthèse	Présenter le projet, ses limites, son architecture et les prochaines étapes.

Routine quotidienne de 30 à 45 minutes

- 10 minutes de lecture.
- 10 minutes pour reformuler le concept sans notes.
- 10 minutes d’exercice ou de schéma.
- 5 à 15 minutes pour tester et noter une erreur ou une question.

Le but n’est pas de mémoriser des acronymes. C’est de pouvoir expliquer comment les composants s’articulent et quelles limites ils introduisent.

39. Ressources vidéo et documentation

Liens pour approfondir

Les liens ci-dessous privilégient des ressources connues ou des recherches YouTube ciblées. Les résultats d'une page de recherche peuvent changer : vérifie le titre, la date, le niveau et la qualité de la vidéo avant de la suivre.

Réseaux de neurones — 3Blue1Brown
Visualisation intuitive des réseaux de neurones et de l'apprentissage.
<link href='https://www.youtube.com/watch?v=aircAruvnKk' color='#2563EB'>https://www.youtube.com/watch?v=aircAruvnKk</link>

Neural networks — recherche YouTube
Autres explications visuelles sur les neurones artificiels.
<link href='https://www.youtube.com/results?search_query=artificial+neural+network+explained+visual' color='#2563EB'>https://www.youtube.com/results?search_query=artificial+neural+network+explained+visual</link>

Transformer et attention — recherche YouTube
Comprendre l'attention et l'architecture Transformer.
<link href='https://www.youtube.com/results?search_query=transformer+attention+explained+for+beginners' color='#2563EB'>https://www.youtube.com/results?search_query=transformer+attention+explained+for+beginners</link>

Andrej Karpathy — Let's build GPT
Démonstration technique de la construction d'un petit modèle de type GPT ; plus avancée.
<link href='https://www.youtube.com/watch?v=kCc8FmEb1nY' color='#2563EB'>https://www.youtube.com/watch?v=kCc8FmEb1nY</link>

MCP — documentation officielle
Concepts, architecture et guides du Model Context Protocol.
<link href='https://modelcontextprotocol.io/introduction' color='#2563EB'>https://modelcontextprotocol.io/introduction</link>

MCP — vidéos explicatives
Recherche YouTube sur les serveurs MCP, les clients et les outils.
<link href='https://www.youtube.com/results?search_query=Model+Context+Protocol+MCP+server+explained' color='#2563EB'>https://www.youtube.com/results?search_query=Model+Context+Protocol+MCP+server+explained</link>

API pour débutants — recherche YouTube
Requêtes, réponses HTTP, JSON et clés API.
<link href='https://www.youtube.com/results?search_query=API+explained+for+beginners+HTTP+JSON' color='#2563EB'>https://www.youtube.com/results?search_query=API+explained+for+beginners+HTTP+JSON</link>

RAG — recherche YouTube
Récupération documentaire et génération augmentée.
<link href='https://www.youtube.com/results?search_query=RAG+retrieval+augmented+generation+explained' color='#2563EB'>https://www.youtube.com/results?search_query=RAG+retrieval+augmented+generation+explained</link>

Prompt engineering — recherche YouTube
Méthodes de formulation, exemples et évaluation de prompts.
<link href='https://www.youtube.com/results?search_query=prompt+engineering+for+beginners+examples' color='#2563EB'>https://www.youtube.com/results?search_query=prompt+engineering+for+beginners+examples</link>

Hugging Face — cours NLP
Ressources pédagogiques sur les modèles de langage et le traitement automatique du langage.
<link href='https://huggingface.co/learn/nlp-course/chapter1/1' color='#2563EB'>https://huggingface.co/learn/nlp-course/chapter1/1</link>

Google Machine Learning Crash Course
Cours structuré sur les bases du machine learning.
<link href='https://developers.google.com/machine-learning/crash-course' color='#2563EB'>https://developers.google.com/machine-learning/crash-course</link>

OpenAI — documentation API
Concepts et guides pour utiliser des modèles via API.
<link href='https://platform.openai.com/docs/' color='#2563EB'>https://platform.openai.com/docs/</link>

Conseil : regarde une vidéo avec un objectif précis. Note trois idées, un exemple et une question. Ensuite, explique le sujet sans reprendre mot pour mot la vidéo.

40. Fiche de révision — Les grandes distinctions

Les confusions à éviter

Ne pas confondre	Différence clé
IA et LLM	L’IA est un domaine large ; un LLM est une famille de modèles de langage.
Entraînement et inférence	L’un ajuste les paramètres ; l’autre utilise le modèle entraîné.
Modèle et application	Le modèle produit des sorties ; l’application gère aussi l’interface, les données, les outils et les règles.
Contexte et mémoire	Le contexte est fourni à la requête ; une mémoire persistante dépend d’un mécanisme de stockage.
API et MCP	Une API expose un contrat de service ; MCP standardise l’échange entre applications compatibles et serveurs de capacités.
Serveur MCP et agent	Le serveur expose des capacités ; l’agent décide éventuellement quand les utiliser.
Skill et outil	Une skill décrit une méthode réutilisable ; un outil exécute une opération concrète.
RAG et fine-tuning	Le RAG récupère des données externes au moment de la requête ; le fine-tuning adapte les paramètres du modèle.
Recherche et génération	La recherche retrouve des contenus ; la génération produit une réponse à partir du contexte et du modèle.
Poids ouverts et logiciel libre	L’accès aux paramètres ne signifie pas automatiquement que toutes les composantes sont ouvertes.
Réponse plausible et réponse vraie	La fluidité n’est pas une preuve ; les affirmations importantes doivent être vérifiées.

Si tu sais expliquer chacune de ces différences avec un exemple, tu possèdes déjà une base solide pour discuter d’architecture IA.

41. Glossaire complémentaire

D’autres termes utiles

Fine-tuning supervisé — Adaptation d’un modèle avec des exemples d’entrées et de sorties souhaitées.

Quantification — Réduction de la précision numérique utilisée pour représenter certains paramètres, souvent pour diminuer mémoire et coût.

Distillation — Technique qui utilise un modèle enseignant pour aider à entraîner un modèle plus petit.

Cache — Stockage temporaire de résultats afin d’éviter de refaire certains calculs.

Prompt système — Instructions de niveau élevé fournies par l’application au modèle, selon l’architecture du produit.

Sortie structurée — Réponse contrainte par un format, par exemple un schéma JSON.

Schéma — Description formelle de la structure attendue des données.

Guardrails — Contrôles qui encadrent les entrées, sorties ou actions d’un système.

Human-in-the-loop — Étape où une personne valide, corrige ou supervise le résultat.

Drift — Évolution des données ou du contexte d’usage qui peut dégrader les performances.

A/B test — Comparaison de variantes sur des groupes ou conditions définis.

Observabilité — Journaux, métriques et traces permettant de comprendre le système.

Idempotence — Propriété d’une opération qui peut être répétée sans produire plusieurs fois un effet supplémentaire indésirable.

Webhook — Mécanisme par lequel un service envoie une notification à une adresse lorsqu’un événement survient.

OAuth — Famille de standards d’autorisation permettant à un service d’accéder à certaines ressources selon des permissions.

Secret manager — Système conçu pour stocker et distribuer des secrets de manière contrôlée.

Rate limiting — Limitation du nombre d’appels sur une période donnée.

Fallback — Solution de repli utilisée quand le chemin principal échoue.

Evaluation set — Ensemble d’exemples utilisés pour mesurer la qualité d’un système.

Ground truth — Référence considérée comme correcte pour une tâche d’évaluation.

Ce glossaire complémentaire n’est pas à mémoriser d’un coup. Utilise-le comme référence lorsque tu rencontres ces termes dans la documentation ou les projets.

42. Exemple de prompt complet pour apprendre

Un tuteur IA avec méthode

Tu peux réutiliser le prompt ci-dessous pour apprendre un sujet technique sans te noyer dans le jargon.

PROMPT À COPIER

« Tu es mon tuteur pédagogique. Je débute sur le sujet suivant : [SUJET]. Mon objectif est de pouvoir l'expliquer et l'utiliser, pas seulement de mémoriser une définition. Procède ainsi :

1. Donne une définition simple en deux phrases.
2. Explique le mécanisme étape par étape.
3. Donne une analogie et précise où elle cesse d'être exacte.
4. Donne un exemple concret.
5. Définis les termes techniques nécessaires.
6. Présente trois erreurs de débutant.
7. Pose-moi trois questions, une à la fois, et attends mes réponses.
8. Corrige mes réponses avec précision et explique mes erreurs.
9. Termine par une fiche de révision et un exercice pratique.

N'invente pas de faits pour combler les lacunes. Si le sujet dépend d'informations récentes, indique ce qu'il faudrait vérifier dans une source à jour. »

Pourquoi ce prompt fonctionne mieux qu'une demande vague

- Il définit un objectif d'apprentissage observable.
- Il impose une progression pédagogique.
- Il distingue l'analogie de la réalité technique.
- Il inclut un test actif, plus utile que la lecture passive.
- Il prévoit l'incertitude et la vérification.

Conseil : change le niveau de détail selon le sujet. Une demande très longue n'est pas toujours meilleure ; la structure doit servir l'objectif.

43. Exemple de spécification d’un assistant IA

Avant de coder, écrire ce que le système doit faire

Une spécification courte permet de rendre le projet testable. Elle évite de commencer par une pile technologique choisie au hasard.

Rubrique	Exemple
Problème	Les employés perdent du temps à chercher les procédures internes.
Utilisateurs	Employés autorisés à consulter les procédures.
Entrées	Question en langage naturel.
Sources	Documents validés, datés et associés à des permissions.
Sortie	Réponse courte, liens vers les sources, incertitude si nécessaire.
Hors périmètre	Conseils juridiques définitifs, décisions RH, accès non autorisés.
Critère de réussite	Réponses soutenues par les documents et taux d’erreur inférieur au seuil fixé.
Escalade	Si la source manque ou si la demande est sensible, orienter vers un humain.
Contraintes	Budget, latence, confidentialité, disponibilité.
Mesure	Qualité de recherche, exactitude, taux de refus appropriés, coût par tâche.

Avant de construire

Fais relire cette spécification par les personnes qui utiliseront le système. Les attentes réelles, les exceptions et les risques apparaissent souvent avant le choix du modèle.

44. Architecture type : assistant avec RAG et outils

Comprendre le flux de bout en bout

Voici un flux conceptuel. Une architecture réelle peut différer, mais le schéma aide à voir les responsabilités.

Étape	Composant	Responsabilité
1	Frontend	Reçoit la question et affiche le résultat.
2	Backend	Authentifie, valide et orchestre.
3	Recherche / RAG	Trouve les passages autorisés les plus pertinents.
4	Modèle	Interprète le contexte et génère une réponse.
5	Outils	Effectuent des opérations externes explicitement autorisées.
6	Contrôles	Valident permissions, schéma, sources et actions sensibles.
7	Observabilité	Mesure qualité, erreurs, latence et coût.
8	Interface	Présente réponse, sources, incertitudes et options de recours.

Où MCP peut-il intervenir ?

MCP peut fournir une interface standard pour découvrir et appeler certains outils ou accéder à certaines ressources. Il ne remplace pas le backend, la recherche, les contrôles d'autorisation ni l'évaluation. Le client et le serveur doivent toujours être configurés de manière sûre.

Où une skill peut-elle intervenir ?

Une skill peut décrire la procédure de traitement : comment clarifier une question, quels critères appliquer, quel format produire et quand escalader. Les actions réelles doivent néanmoins être exécutées par des composants qui vérifient les droits.

La séparation des responsabilités rend le système plus compréhensible et plus facile à tester.

45. Checklist personnelle : suis-je prêt ?

Auto-évaluation finale

Coche chaque ligne lorsque tu peux l’expliquer sans lire le manuel.

- ■ Je peux expliquer la différence entre IA, machine learning et deep learning.
- ■ Je peux décrire les grandes étapes de l’entraînement d’un réseau de neurones.
- ■ Je comprends la différence entre entraînement et inférence.
- ■ Je peux expliquer simplement ce qu’est un LLM et un token.
- ■ Je distingue modèle, application, frontend, backend et serveur.
- ■ Je peux décrire une requête API et interpréter quelques erreurs courantes.
- ■ Je comprends à quoi servent les embeddings et la recherche vectorielle.
- ■ Je peux expliquer le principe du RAG et ses limites.
- ■ Je sais distinguer API, outil, skill, workflow, agent et MCP.
- ■ Je peux expliquer les rôles du client et du serveur MCP.
- ■ Je sais structurer un prompt avec objectif, contexte, contraintes et format.
- ■ Je vérifie les affirmations importantes au lieu de faire confiance au ton.
- ■ Je sais identifier les données sensibles et limiter les permissions.
- ■ Je peux définir des tests et des critères de réussite.
- ■ Je sais expliquer pourquoi une solution simple peut être préférable à un agent autonome.

Interprétation

- 0–5 : consolide les fondamentaux et refais les premiers exercices.
- 6–10 : tu peux commencer un petit projet guidé.
- 11–13 : tu comprends plusieurs briques et peux concevoir une architecture simple.
- 14–15 : tu as une base solide ; continue avec un projet réel et une évaluation rigoureuse.

Cette auto-évaluation mesure une compréhension introductive, pas une qualification professionnelle. Les systèmes réels exigent des compétences supplémentaires selon les risques et le domaine.

46. Corrigé indicatif — questions de compréhension

Exemples de réponses attendues

1. IA, machine learning, deep learning

L’IA est le domaine général. Le machine learning est une famille de méthodes qui apprennent à partir de données. Le deep learning est une famille du machine learning fondée sur des réseaux de neurones à plusieurs couches.

2. Entraînement et inférence

L’entraînement ajuste les paramètres à partir d’exemples. L’inférence utilise les paramètres existants pour produire une sortie pour une nouvelle entrée.

3. API et MCP

Une API décrit comment appeler un service. MCP est un protocole qui standardise la manière dont des applications compatibles peuvent découvrir et utiliser des capacités exposées par des serveurs. Un serveur MCP peut lui-même s’appuyer sur des API.

4. RAG et fine-tuning

Le RAG récupère des informations externes au moment de la requête et les ajoute au contexte. Le fine-tuning adapte les paramètres d’un modèle à partir d’un nouvel entraînement. Les deux approches peuvent être combinées.

5. Pourquoi vérifier ?

Un modèle génère une réponse plausible à partir du contexte et de ses paramètres, mais cela ne garantit pas que chaque affirmation soit exacte. La vérification est essentielle lorsque l’erreur a des conséquences.

Pour les exercices de conception, plusieurs solutions peuvent être correctes. La qualité dépend de la clarté du besoin, des risques anticipés et de la capacité à tester le système.

47. Construire une culture de travail avec l’IA

Un cadre professionnel

L’usage professionnel de l’IA doit être organisé autour du résultat, de la qualité et de la responsabilité. L’objectif n’est pas d’ajouter de l’IA à chaque tâche, mais de l’utiliser là où elle apporte une amélioration mesurable.

Cadre de travail recommandé

1. Définir la tâche et le niveau de risque.
2. Déterminer ce que l’IA peut faire et ce qui reste sous responsabilité humaine.
3. Fournir des données adaptées et autorisées.
4. Choisir un modèle ou un outil sur des critères explicites.
5. Définir une sortie attendue et une grille d’évaluation.
6. Vérifier les résultats importants.
7. Documenter les limites, les décisions et les changements.
8. Réviser régulièrement le système à mesure que les modèles, données ou besoins évoluent.

Questions à poser à chaque projet

- Quel problème précis est résolu ?
- Comment sait-on que le résultat est meilleur qu’avant ?
- Quel est le coût total, y compris le contrôle humain ?
- Que se passe-t-il quand le modèle échoue ?
- Qui est responsable de la décision finale ?
- Quelles données sont envoyées et à qui ?
- Comment peut-on arrêter ou revenir en arrière ?

La compétence la plus importante n’est pas de connaître tous les outils. C’est de savoir choisir la bonne approche, comprendre ses limites et vérifier le résultat.

48. Plan de progression après ce manuel

Du débutant au constructeur de systèmes

Après les bases, choisis une direction plutôt que d’essayer de tout apprendre simultanément.

Piste	À apprendre ensuite	Projet conseillé
Utilisateur avancé	Prompts structurés, recherche web, vérification, documents et automatisations simples.	Assistant personnel de recherche avec sources.
Développeur IA	API, JSON, gestion des erreurs, embeddings, RAG, tests et observabilité.	Assistant documentaire avec évaluation.
Automatisation	Workflows, webhooks, permissions, connecteurs et traitement des exceptions.	Tri et résumé de demandes avec validation humaine.
Agents et MCP	Découverte d’outils, schémas, permissions, orchestration et sécurité.	Assistant qui consulte un service en lecture seule.
Machine learning	Python, algèbre linéaire, probabilités, optimisation et évaluation.	Classificateur simple entraîné sur un jeu de données.
Gouvernance IA	Protection des données, risques, documentation, tests et politiques d’usage.	Évaluation des risques d’un assistant interne.

Ordre recommandé

1) Maîtriser les concepts. 2) Construire un prototype simple. 3) Mesurer sa qualité. 4) Ajouter la récupération documentaire ou les outils si nécessaire. 5) Tester les permissions et les erreurs. 6) Envisager l’autonomie seulement lorsque les contrôles sont suffisants.

49. Sources de référence et précautions

Privilégier les documents primaires

Les technologies d’IA évoluent vite. Les concepts fondamentaux restent utiles, mais les noms de fonctionnalités, les prix, les capacités et les paramètres des produits changent. Pour une décision technique, consulte toujours la documentation actuelle du fournisseur concerné.

Références utiles

Model Context Protocol — introduction
<link href='https://modelcontextprotocol.io/introduction' color='#2563EB'>https://modelcontextprotocol.io/introduction</link>

Google Machine Learning Crash Course
<link href='https://developers.google.com/machine-learning/crash-course' color='#2563EB'>https://developers.google.com/machine-learning/crash-course</link>

Hugging Face NLP Course
<link href='https://huggingface.co/learn/nlp-course/chapter1/1' color='#2563EB'>https://huggingface.co/learn/nlp-course/chapter1/1</link>

Documentation OpenAI API
<link href='https://platform.openai.com/docs/' color='#2563EB'>https://platform.openai.com/docs/</link>

3Blue1Brown — réseaux de neurones
<link href='https://www.youtube.com/watch?v=aircAruvnKk' color='#2563EB'>https://www.youtube.com/watch?v=aircAruvnKk</link>

Andrej Karpathy — construction d’un GPT
<link href='https://www.youtube.com/watch?v=kCc8FmEb1nY' color='#2563EB'>https://www.youtube.com/watch?v=kCc8FmEb1nY</link>

Note sur les vidéos

Les vidéos citées peuvent être en anglais et leur niveau varie. Active les sous-titres si nécessaire. Pour MCP, privilégie les contenus récents et vérifie qu’ils correspondent à la version actuelle du protocole.

Note de méthode

Ce manuel est un support pédagogique général, pas une certification ni un conseil juridique, médical ou de sécurité informatique adapté à un cas particulier. Pour déployer un système en production, consulte les documentations officielles et les spécialistes appropriés.

50. Conclusion — comprendre avant d’automatiser

La carte mentale à conserver

L’intelligence artificielle devient plus facile à comprendre lorsqu’on sépare les couches du système.

Couche	Question essentielle
Modèle	Comment transforme-t-il les entrées en sorties ?
Prompt et contexte	Quelles instructions et informations reçoit-il ?
Données / RAG	Quelles sources externes peuvent être récupérées ?
Outils / API / MCP	Quelles actions ou données le système peut-il utiliser ?
Backend / orchestration	Qui coordonne les étapes, les permissions et les erreurs ?
Frontend	Comment l’utilisateur interagit-il avec le système ?
Évaluation / sécurité	Comment vérifier la qualité, protéger les données et limiter les risques ?

Les cinq principes à retenir

1. Une IA est un système probabiliste et technique, pas une source infaillible.
2. Un modèle n’est qu’une brique : les données, les outils et l’application comptent autant.
3. Un prompt clair améliore la demande, mais ne remplace pas la vérification.
4. L’autonomie doit être limitée par les permissions, les tests et le niveau de risque.
5. La meilleure solution est celle qui résout le problème avec une qualité mesurable, un coût acceptable et des risques maîtrisés.

Si tu peux expliquer ces principes à quelqu’un qui débute, dessiner une architecture simple et construire un petit assistant que tu sais tester, tu as franchi une étape importante : tu ne fais plus seulement « utiliser l’IA », tu commences à comprendre comment les systèmes d’IA fonctionnent.

Fin du manuel — garde-le comme document vivant : ajoute tes propres exemples, tes erreurs, tes tests et les ressources que tu valides.